

# ml2

September 29, 2025

## 0.1 Ejercicio 2: clustering con K-means

### 1 IMPORTS

```
[9]: %matplotlib inline
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from scipy.spatial.distance import cdist
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import robust_scale
from sklearn.model_selection import learning_curve
```

#### 1.0.1

Realizamos las lecturas de clusters y representamos los resultados

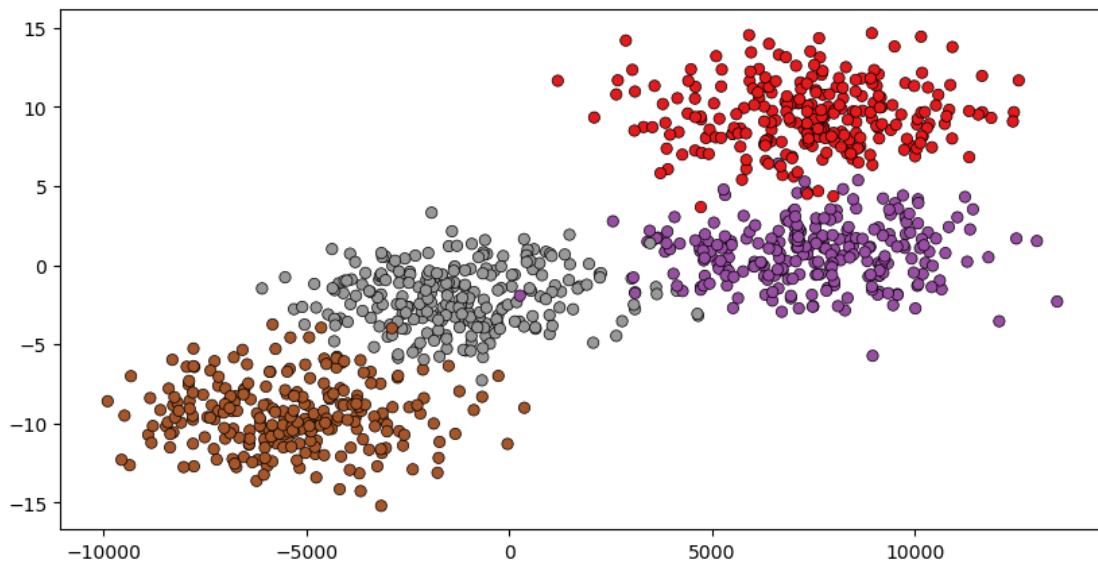
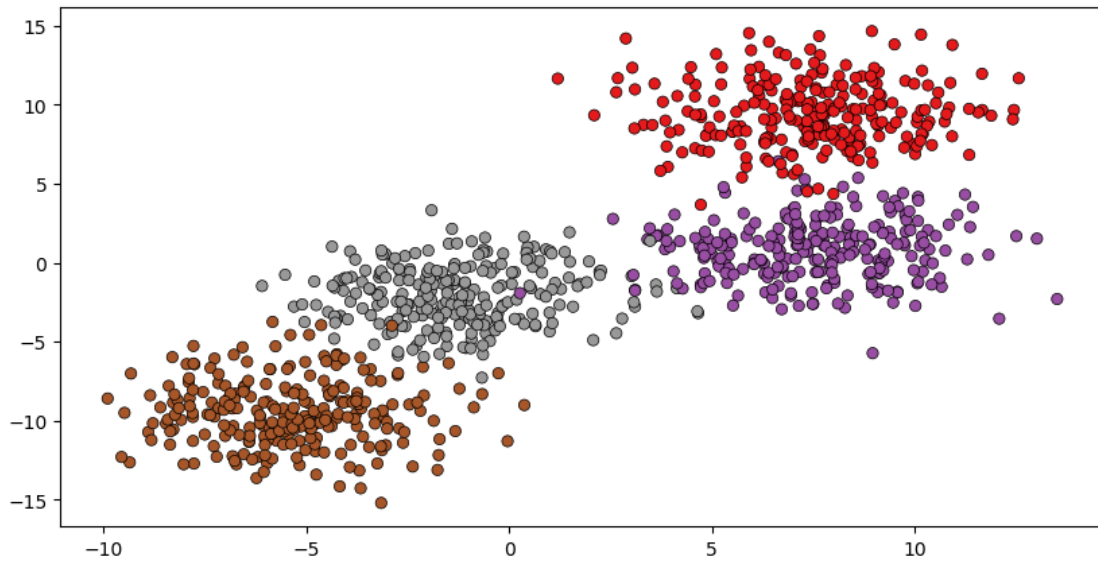
```
[10]: df1 = pd.read_csv("clust_ex1.csv")
df = pd.read_csv("clust_ex2.csv")
print(df.head())
print(df1.head())
```

|   | x1        | x2        | y |
|---|-----------|-----------|---|
| 0 | 8.422981  | 1.436159  | 1 |
| 1 | 5.518084  | 8.325260  | 0 |
| 2 | -3.919422 | 0.722993  | 3 |
| 3 | -6.137162 | -9.758148 | 2 |
| 4 | 8.275729  | -2.854448 | 1 |

|   | x1           | x2        | y |
|---|--------------|-----------|---|
| 0 | 8422.981343  | 1.436159  | 1 |
| 1 | 5518.083686  | 8.325260  | 0 |
| 2 | -3919.421789 | 0.722993  | 3 |
| 3 | -6137.162076 | -9.758148 | 2 |
| 4 | 8275.729340  | -2.854448 | 1 |

```
[11]: for df in [df, df1]:
      plt.figure(figsize=(10,5))
      plt.scatter(df['x1'],df['x2'],c=df['y'],cmap=plt.cm.Set1,linewidths=0.
↪5,edgecolors='k')
      plt.show()
```



```
[12]: ###Calculamos distancia euclidiana
      dist = cdist([[0,0], [1,1], [2,2]],[[0,0],[1,0]], 'euclidean')
      print(dist)
```

```
np.argmin(dist, axis=1)
```

```
[[0.          1.          ]  
 [1.41421356  1.          ]  
 [2.82842712  2.23606798]]
```

```
[12]: array([0, 1, 1])
```

## 2 Elegimos K

```
[13]: K= 4
```

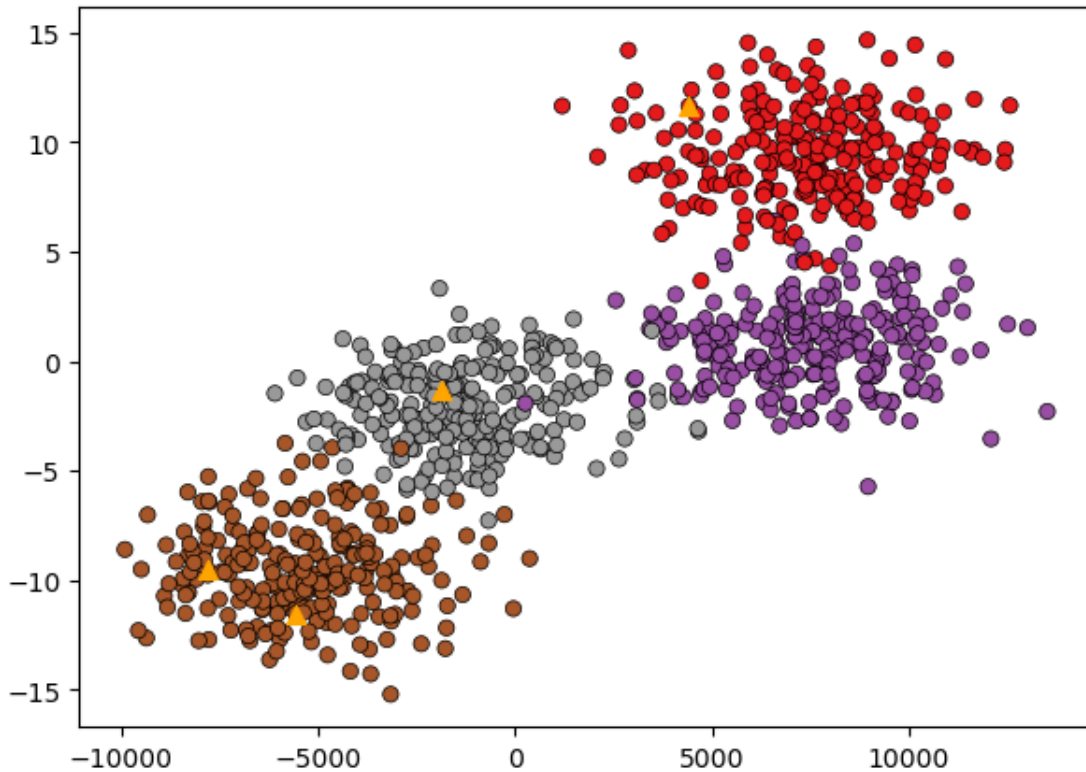
```
[14]: features = ['x1', 'x2']  
centroids = df[features].sample(n=K).values  
centroids
```

```
[14]: array([[ -5.56286887e+03, -1.15124140e+01],  
             [ 4.41552366e+03,  1.16445534e+01],  
             [-7.80004480e+03, -9.51771579e+00],  
             [-1.86808721e+03, -1.35785045e+00]])
```

### 2.0.1

Volvemos a representar con los centroides

```
[15]: plt.figure(figsize=(7,5))  
plt.scatter(df['x1'],df['x2'],c=df['y'],cmap=plt.cm.Set1,linewidths=0.  
↪5,edgecolors='k')  
plt.scatter(centroids[:,0], centroids[:,1], marker='^', s=50, color='orange')  
plt.show()
```



```
[16]: #etiquetamos cada punto con el centroide más cercano
distances = cdist(df[features], centroids, 'euclidean')
distances
```

```
[16]: array([[13985.85620419,  4007.47068869, 16223.02984437, 10291.06893627],
 [11080.97030982,  1102.56502551, 13318.14044144,  7386.17724672],
 [ 1643.49262378,  8334.95260066,  3880.63652681,  2051.33563016],
 ...,
 [ 1642.92105716, 11621.33803109,   594.26148875,  5337.71370873],
 [15095.28379622,  5116.87826017, 17332.45572477, 11400.49296939],
 [ 2685.01651486, 12663.4229411 ,   447.84024901,  6379.80012084]])
```

```
[17]: labels = np.argmin(distances, axis=1)
labels[:10]
df['labels'] = labels
df.head()
```

```
[17]:
```

|   | x1           | x2        | y | labels |
|---|--------------|-----------|---|--------|
| 0 | 8422.981343  | 1.436159  | 1 | 1      |
| 1 | 5518.083686  | 8.325260  | 0 | 1      |
| 2 | -3919.421789 | 0.722993  | 3 | 0      |
| 3 | -6137.162076 | -9.758148 | 2 | 0      |

```
4 8275.729340 -2.854448 1 1
```

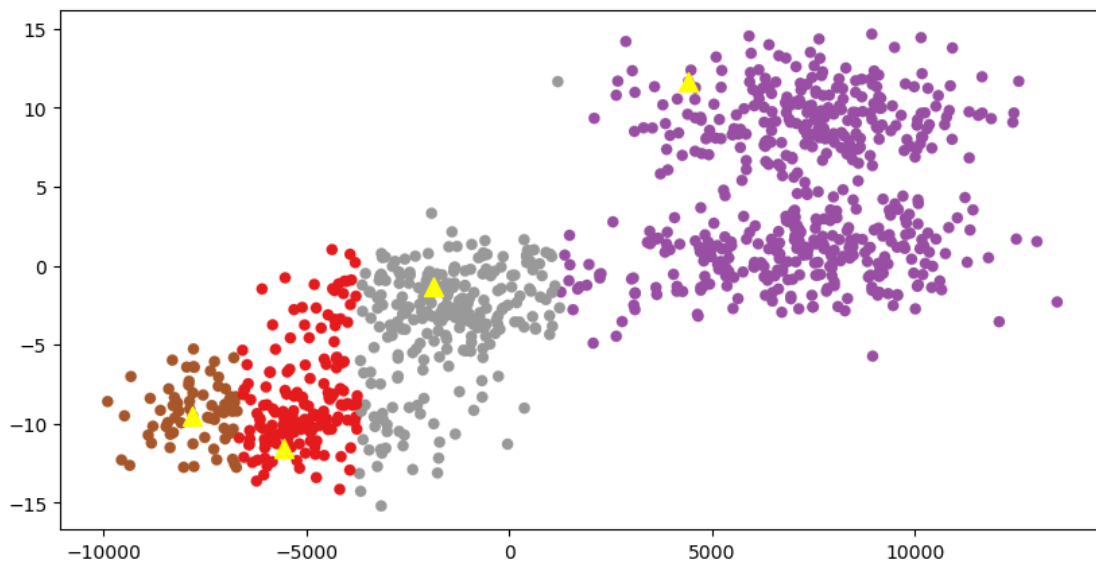
```
[18]: df.drop('y', axis=1, inplace=True)
df.head()
```

```
[18]:
```

|   | x1           | x2        | labels |
|---|--------------|-----------|--------|
| 0 | 8422.981343  | 1.436159  | 1      |
| 1 | 5518.083686  | 8.325260  | 1      |
| 2 | -3919.421789 | 0.722993  | 0      |
| 3 | -6137.162076 | -9.758148 | 0      |
| 4 | 8275.729340  | -2.854448 | 1      |

```
[19]: plt.figure(figsize=(10,5))
plt.scatter(df['x1'],df['x2'],c=df['labels'],cmap=plt.cm.Set1,linewidths=0.1)

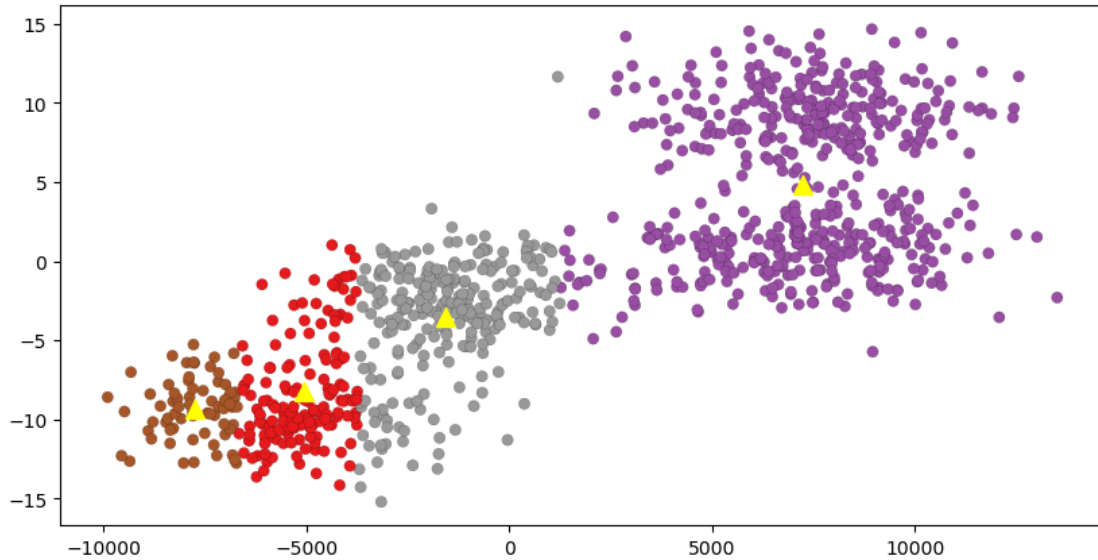
plt.scatter(centroids[:,0], centroids[:,1], marker='^', s=100,color='yellow')
plt.show()
```



```
[20]: #Calculamos los nuevos centroides como el valor medio del conjunto de puntos de
      → cada cluster asignado en la primera iteración
centroids = df.groupby('labels')[features].mean().values
```

```
[21]: plt.figure(figsize=(10,5))
plt.scatter(df['x1'],df['x2'],c=df['labels'],cmap=plt.cm.Set1,linewidths=0.1,
      → edgecolors='k')

plt.scatter(centroids[:,0], centroids[:,1], marker='^', s=100,color='yellow')
plt.show()
```



```
[22]: #Inicializamos el algoritmo KMeans de sklearn con 4 clusters
kmeans = KMeans(n_clusters=4, random_state=33)
#Ajustamos el modelo a nuestro conjunto de datos
kmeans.fit(df[features])
#Añadimos las etiquetas de cluster a nuestro conjunto de datos
df['labels'] = kmeans.predict(df[features])
df.head()
```

```
[22]:
```

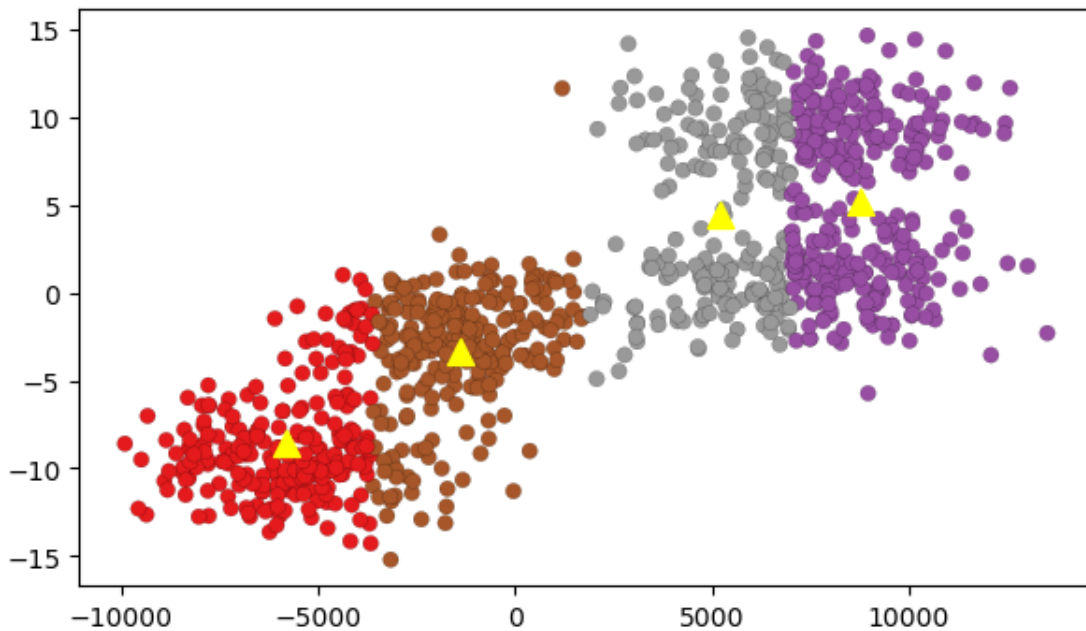
|   | x1           | x2        | labels |
|---|--------------|-----------|--------|
| 0 | 8422.981343  | 1.436159  | 1      |
| 1 | 5518.083686  | 8.325260  | 3      |
| 2 | -3919.421789 | 0.722993  | 0      |
| 3 | -6137.162076 | -9.758148 | 0      |
| 4 | 8275.729340  | -2.854448 | 1      |

```
[23]: #Obtenemos los centroides de los clusters asignados por k-Means
centroids = kmeans.cluster_centers_
centroids
```

```
[23]: array([[ -5.81669002e+03,  -8.56164112e+00],
        [  8.76684892e+03,   5.17585189e+00],
        [ -1.40541895e+03,  -3.33201590e+00],
        [  5.21265672e+03,   4.43732880e+00]])
```

```
[24]: #Visualizamos los clusters y los centroides
plt.figure(figsize=(7,4))
plt.scatter(df['x1'],df['x2'],c=df['labels'],cmap=plt.cm.Set1,linewidths=0.1,
            ↪edgecolors='k')
```

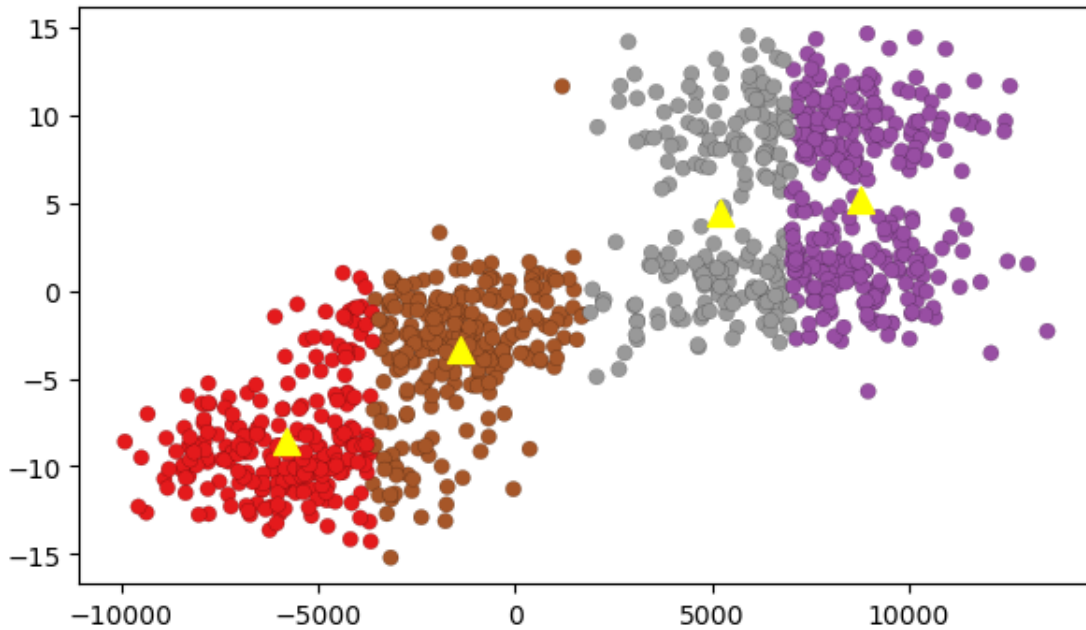
```
plt.scatter(centroids[:,0], centroids[:,1], marker='^', s=100,color='yellow')
plt.show()
```



```
[25]: #Inicializamos el algoritmo KMeans de sklearn con 4 clusters
features = ['x1', 'x2']
kmeans = KMeans(n_clusters=4, random_state=33)

#Ajustamos el modelo a nuestro conjunto de datos
kmeans.fit(df[features])
#Añadimos las etiquetas de cluster a nuestro conjunto de datos
df1['labels'] = kmeans.predict(df1[features])
df1.head()
#Visualizamos los clusters y los centroides
centroids = df1.groupby('labels')[features].mean().values
plt.figure(figsize=(7,4))
plt.scatter(df1['x1'],df1['x2'],c=df1['labels'],cmap=plt.cm.Set1,linewidths=0.
↳1, edgecolors='k')

plt.scatter(centroids[:,0], centroids[:,1], marker='^', s=100,color='yellow')
plt.show()
```



## 2.0.2

Inertia

```
[26]: kmeans.inertia_
```

```
[26]: 1887556620.1150856
```

```
[27]: scaler = MinMaxScaler(feature_range=(0, 1),copy=True)
m = scaler.fit(df1[['x1','x2','labels']])
print(scaler.data_max_)
aux = scaler.transform(df1[['x1','x2','labels']])
print(aux)
df2: pd.DataFrame = pd.DataFrame(df1, columns=['x1', 'x2','labels'])
df2[['x1','x2','labels']] = aux
print(df2.head())
```

```
[1.35048262e+04 1.46566991e+01 3.00000000e+00]
[[0.78282248 0.55725929 0.33333333]
 [0.65867888 0.78796737 1.         ]
 [0.25535802 0.53337619 0.         ]
 ...
 [0.11491193 0.09785276 0.         ]
 [0.8302346  0.80628189 0.33333333]
 [0.07037705 0.22977356 0.         ]]
      x1      x2  labels
0  0.782822  0.557259  0.333333
```



```

1  0.658679  0.787967  1.000000
2  0.255358  0.533376  0.000000
3  0.160581  0.182375  0.000000
4  0.776530  0.413572  0.333333

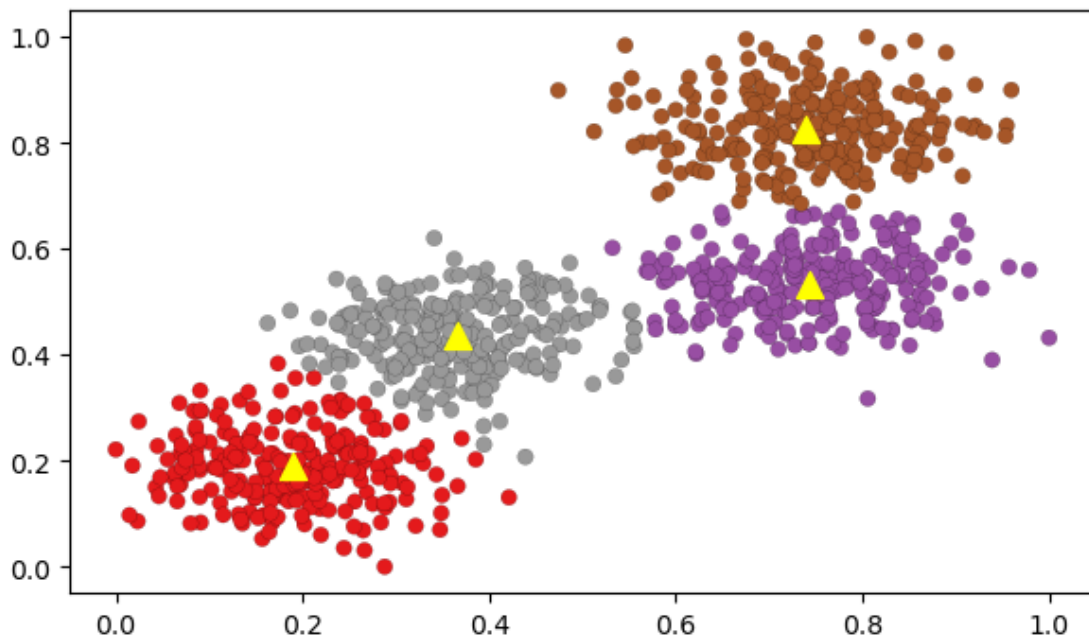
```

```

[28]: #Inicializamos el algoritmo KMeans de sklearn con 4 clusters
features = ['x1', 'x2']
kmeans = KMeans(n_clusters=4, random_state=33)
#Ajustamos el modelo a nuestro conjunto de datos
kmeans.fit(df2[features])
#Añadimos las etiquetas de cluster a nuestro conjunto de datos
df2['labels'] = kmeans.predict(df2[features])
df2.head()
#Visualizamos los clusters y los centroides
centroids = df2.groupby('labels')[features].mean().values
plt.figure(figsize=(7,4))
plt.scatter(df2['x1'],df2['x2'],c=df2['labels'],cmap=plt.cm.Set1,linewidths=0.
↪1, edgecolors='k')

plt.scatter(centroids[:,0], centroids[:,1], marker='^', s=100,color='yellow')
plt.show()

```



### 2.0.3

Comprobamos Inertia

```
[29]: kmeans.inertia_
```

```
[29]: 11.457686114091901
```

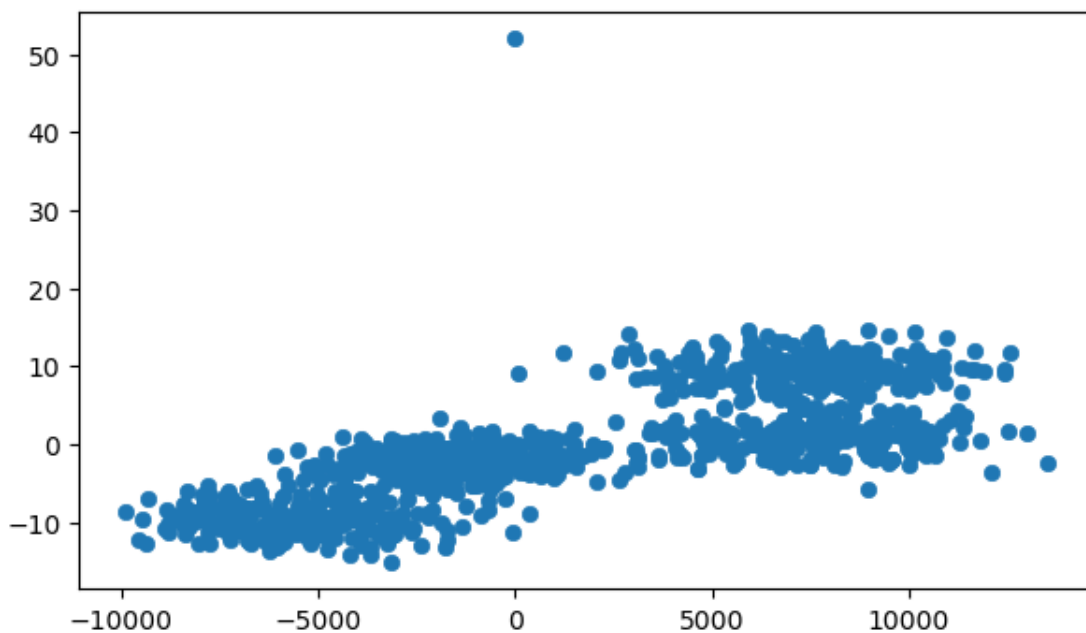
```
[30]: df = pd.read_csv("clust_ex2.csv")
df= pd.read_csv("clust_ex1.csv")
#Creamos valores atípicos y los añadimos al conjunto de datos
df_outliers = pd.DataFrame({'x1': [100, 4, 103], 'x2': [0, 52, 9]})
df = pd.concat([df, df_outliers])
df.head()
```

```
[30]:
```

|   | x1           | x2        | y   |
|---|--------------|-----------|-----|
| 0 | 8422.981343  | 1.436159  | 1.0 |
| 1 | 5518.083686  | 8.325260  | 0.0 |
| 2 | -3919.421789 | 0.722993  | 3.0 |
| 3 | -6137.162076 | -9.758148 | 2.0 |
| 4 | 8275.729340  | -2.854448 | 1.0 |

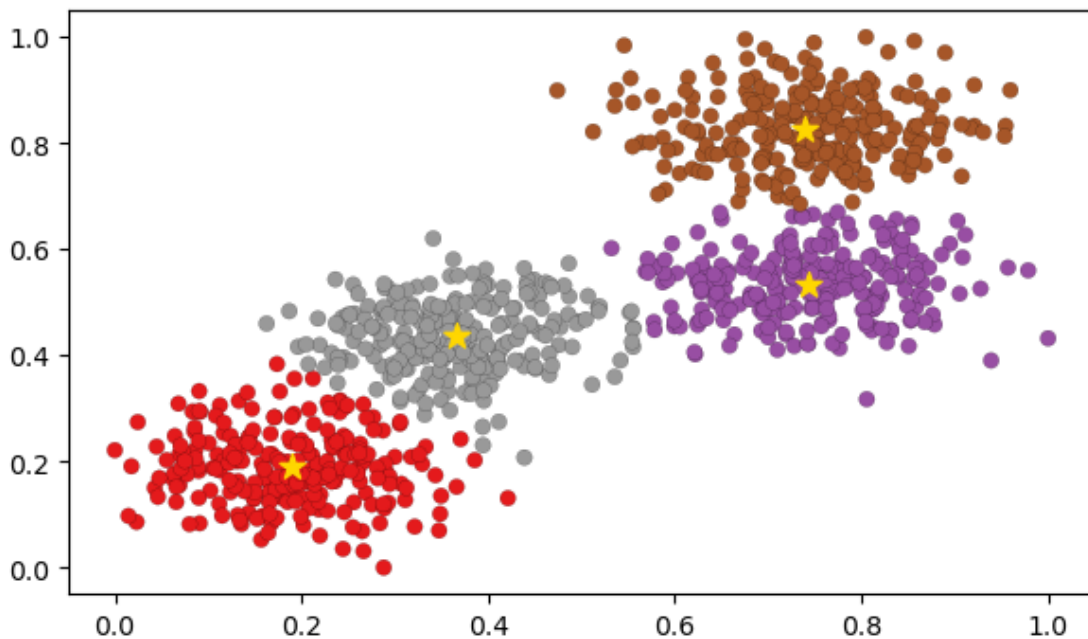
```
[31]: #Visualizamos los datos
plt.figure(figsize=(7,4))
plt.scatter(df['x1'],df['x2'],cmap=plt.cm.Set1,linewidths=0.5)
plt.show()
```

C:\Users\Surface\AppData\Local\Temp\ipykernel\_5544\855068868.py:3: UserWarning:  
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored  
plt.scatter(df['x1'],df['x2'],cmap=plt.cm.Set1,linewidths=0.5)



```
[32]: #Volvemos a realizar el algoritmo KMeans de sklearn con 4 clusters
features = ['x1', 'x2']
kmeans = KMeans(n_clusters=4, random_state=33)
#Ajustamos el modelo a nuestro conjunto de datos
kmeans.fit(df2[features])
#Añadimos las etiquetas de cluster a nuestro conjunto de datos
df2['labels'] = kmeans.predict(df2[features])
df2.head()
#Visualizamos los clusters y los centroides
centroids = df2.groupby('labels')[features].mean().values
plt.figure(figsize=(7,4))
plt.scatter(df2['x1'],df2['x2'],c=df2['labels'],cmap=plt.cm.Set1,linewidths=0.
↪1, edgecolors='k')

plt.scatter(centroids[:,0], centroids[:,1], marker='*', s=100,color='gold')
plt.show()
```



```
[33]: X = df2[["x1", "x2"]].to_numpy()
labels = df2["labels"].to_numpy()
```

```
[34]: #silhouette score
silhouette_avg = silhouette_score(X, labels)
silhouette_avg
```

```
[34]: np.float64(0.5467681733359228)
```

Teniendo en cuenta que el valor es más cercano a 1 que a 0, indica que el punto está bien ubicado en su cluster y está considerablemente separado de otros clusters, lo que sugiere una buena calidad de clustering.

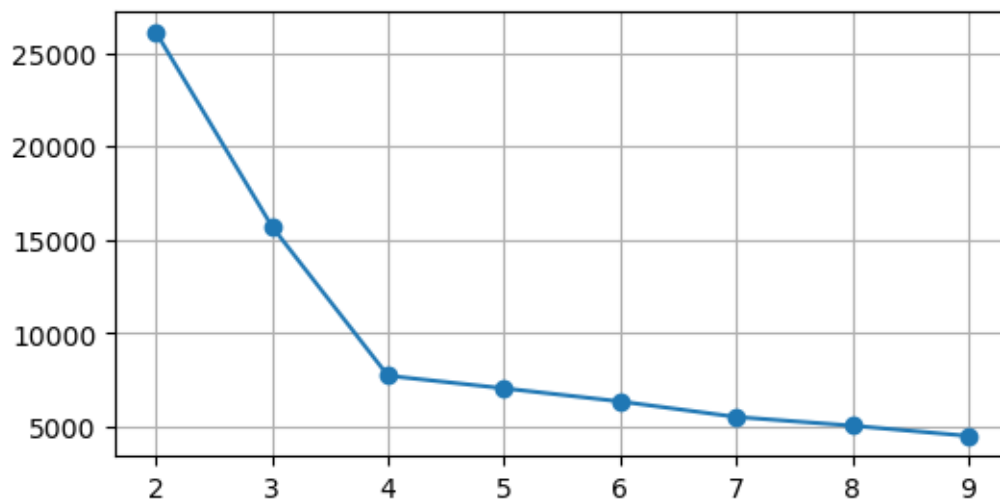
```
[35]: df = pd.read_csv('clust_ex2.csv')
      df1= pd.read_csv('clust_ex1.csv')
      df.head()
      df1.head()
```

```
[35]:      x1      x2  y
0  8422.981343  1.436159  1
1  5518.083686  8.325260  0
2 -3919.421789  0.722993  3
3 -6137.162076 -9.758148  2
4  8275.729340 -2.854448  1
```

```
[36]: Ks = range(2, 10)
```

```
[37]: inertias = []
      for k in Ks:
          kmeans = KMeans(n_clusters=k, random_state=33)
          kmeans.fit(df[features])
          inertias.append(kmeans.inertia_)
```

```
[38]: #representamos elbow
      plt.figure(figsize=(6,3))
      plt.plot(Ks, inertias, marker='o')
      plt.grid()
      plt.show()
```



### 3

Entre k=1 y k=4 hay una gran caída de la inercia (la curva baja muy fuerte).

A partir de k=4 la curva empieza a “aplanarse”

De k=5 en adelante la reducción de inercia es mínima. sobreajuste (clusters que no aportan tanta información nueva).

Como conclusión esto hace indicar que el número de clusters, acorde con segmentación esta en k=4

#### 3.0.1

Importamos las librerías que necesitamos para introducir el algoritmo de clasificación, en este caso sera un **SVM** y realizamos los pasos necesarios para entrenar el modelo

```
[39]: from sklearn import svm
      from sklearn.model_selection import train_test_split
      from sklearn.metrics import classification_report, confusion_matrix,
      accuracy_score
```

```
[40]: df = pd.read_csv("clust_ex2.csv")
      df.head()
```

```
[40]:      x1      x2  y
0  8.422981  1.436159  1
1  5.518084  8.325260  0
2 -3.919422  0.722993  3
3 -6.137162 -9.758148  2
4  8.275729 -2.854448  1
```

```
[41]: x = df[['x1', 'x2']]
      y = df['y']
```

```
[42]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3,
      random_state=33)
```

```
[43]: classifier = svm.SVC(kernel='linear')
      classifier.fit(x_train, y_train)
      y_pred = classifier.predict(x_test)
      print(confusion_matrix(y_test, y_pred))
      print(classification_report(y_test, y_pred))
      print(accuracy_score(y_test, y_pred))
```

```
[[66  1  0  0]
 [ 3 82  0  1]
 [ 0  0 74  0]
 [ 0  1  1 71]]

      precision    recall  f1-score   support


```

|              |      |      |      |     |
|--------------|------|------|------|-----|
| 0            | 0.96 | 0.99 | 0.97 | 67  |
| 1            | 0.98 | 0.95 | 0.96 | 86  |
| 2            | 0.99 | 1.00 | 0.99 | 74  |
| 3            | 0.99 | 0.97 | 0.98 | 73  |
| accuracy     |      |      | 0.98 | 300 |
| macro avg    | 0.98 | 0.98 | 0.98 | 300 |
| weighted avg | 0.98 | 0.98 | 0.98 | 300 |

0.9766666666666667

```
[44]: ### Realizamos una comprobación con una cross validation ###
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import cross_val_score, cross_validate, \
    ↪StratifiedKFold
from sklearn.svm import SVC
from sklearn.metrics import make_scorer, f1_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn import svm
clf = classifier
scores = cross_val_score(clf, x, y, cv=5)
print("Accuracy medio:", np.mean(scores))
print("Accuracy por fold:", scores)
print("f1-score por fold:", cross_val_score(clf, x, y, cv=5, \
    ↪scoring='f1_macro'))
print("precision por fold:", cross_val_score(clf, x, y, cv=5, \
    ↪scoring='precision_macro'))
print("recall por fold:", cross_val_score(clf, x, y, cv=5, \
    ↪scoring='recall_macro'))
```

Accuracy medio: 0.977

Accuracy por fold: [0.97 0.97 0.99 0.985 0.97 ]

f1-score por fold: [0.96994034 0.97004647 0.989999 0.98494899 0.97008111]

precision por fold: [0.97058069 0.97039216 0.99019608 0.984996 0.97112807]

recall por fold: [0.97 0.97 0.99 0.985 0.97 ]

```
[45]: pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", SVC(kernel="linear", C=1, random_state=33))
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=33)

metrics = {
    "Accuracy": "accuracy",
    "F1 (macro)": "f1_macro",
```

```

    "Precision (macro)": "precision_macro",
    "Recall (macro)": "recall_macro",
}

fig, axes = plt.subplots(2, 2, figsize=(10, 7), sharex=True)
axes = axes.ravel()

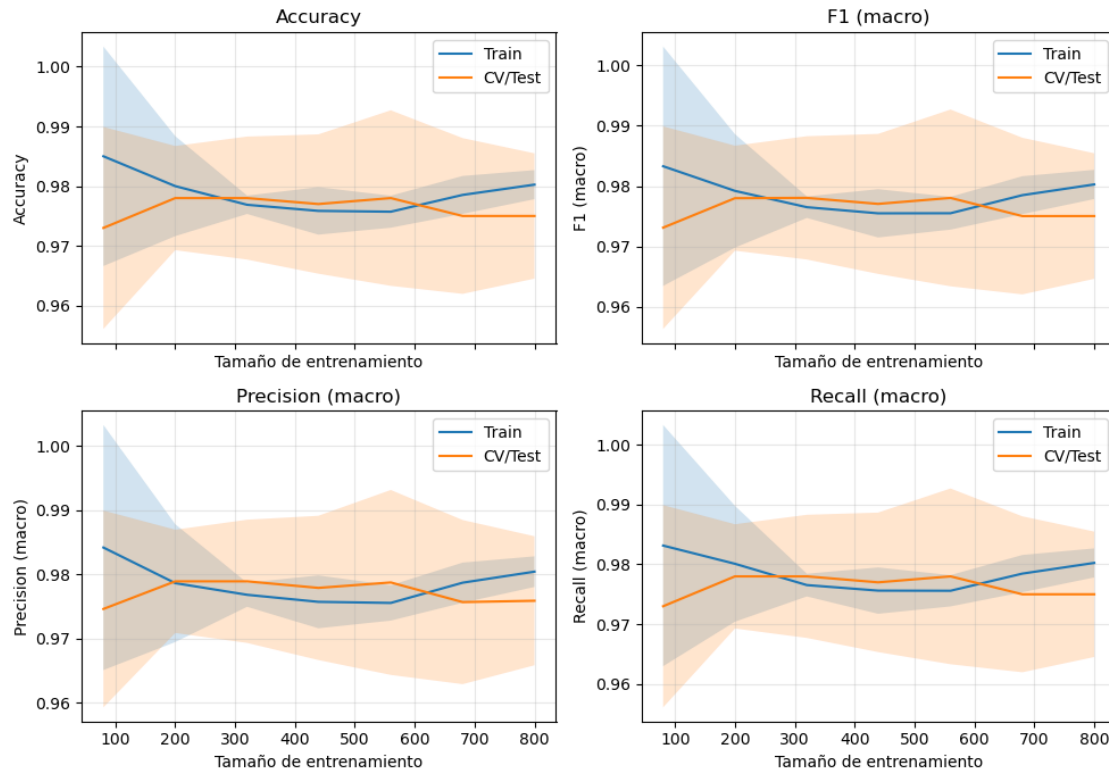
for ax, (title, score_name) in zip(axes, metrics.items()):
    train_sizes, train_scores, test_scores = learning_curve(
        estimator=pipe,
        X=X, y=y,
        cv=cv,
        scoring=score_name,
        n_jobs=-1,
        train_sizes=np.linspace(0.1, 1.0, 7),
        shuffle=True,
        random_state=33
    )
    tr_mean, tr_std = train_scores.mean(axis=1), train_scores.std(axis=1)
    te_mean, te_std = test_scores.mean(axis=1), test_scores.std(axis=1)

    ax.plot(train_sizes, tr_mean, label="Train")
    ax.fill_between(train_sizes, tr_mean - tr_std, tr_mean + tr_std, alpha=0.2)
    ax.plot(train_sizes, te_mean, label="CV/Test")
    ax.fill_between(train_sizes, te_mean - te_std, te_mean + te_std, alpha=0.2)

    ax.set_title(title)
    ax.set_xlabel("Tamaño de entrenamiento")
    ax.set_ylabel(title)
    ax.grid(True, alpha=0.3)
    ax.legend()

plt.tight_layout()
plt.show()

```



### 3.0.2

como podemos comprobar en todas las graficas, el modelo no tiene una gran variación entre el train y el test, ya que se mantiene en valores altos (0.95-1), por lo que considero que el modelo de svm linear funciona bastante y no tiene ningún sobreajuste de los datos. Además el modelo debería responder bien a nuevos datos.

```
[51]: pip install nbconvert
```

```
Requirement already satisfied: nbconvert in c:\users\surface\anaconda3\lib\site-packages (7.16.6)
Requirement already satisfied: beautifulsoup4 in
c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (4.12.3)
Requirement already satisfied: bleach!=5.0.0 in
c:\users\surface\anaconda3\lib\site-packages (from
bleach[css]!=5.0.0->nbconvert) (6.2.0)
Requirement already satisfied: defusedxml in
c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (0.7.1)
Requirement already satisfied: Jinja2>=3.0 in
c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in
c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (5.7.2)
Requirement already satisfied: jupyterlab-pygments in
```



c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (0.3.0)  
 Requirement already satisfied: markupsafe>=2.0 in  
 c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (3.0.2)  
 Requirement already satisfied: mistune<4,>=2.0.3 in  
 c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (3.1.2)  
 Requirement already satisfied: nbclient>=0.5.0 in  
 c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (0.10.2)  
 Requirement already satisfied: nbformat>=5.7 in  
 c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (5.10.4)  
 Requirement already satisfied: packaging in c:\users\surface\anaconda3\lib\site-  
 packages (from nbconvert) (24.2)  
 Requirement already satisfied: pandocfilters>=1.4.1 in  
 c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (1.5.0)  
 Requirement already satisfied: pygments>=2.4.1 in  
 c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (2.19.1)  
 Requirement already satisfied: traitlets>=5.1 in  
 c:\users\surface\anaconda3\lib\site-packages (from nbconvert) (5.14.3)  
 Requirement already satisfied: webencodings in  
 c:\users\surface\anaconda3\lib\site-packages (from  
 bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)  
 Requirement already satisfied: tinycss2<1.5,>=1.1.0 in  
 c:\users\surface\anaconda3\lib\site-packages (from  
 bleach[css]!=5.0.0->nbconvert) (1.4.0)  
 Requirement already satisfied: platformdirs>=2.5 in  
 c:\users\surface\anaconda3\lib\site-packages (from jupyter-core>=4.7->nbconvert)  
 (4.3.7)  
 Requirement already satisfied: pywin32>=300 in  
 c:\users\surface\anaconda3\lib\site-packages (from jupyter-core>=4.7->nbconvert)  
 (308)  
 Requirement already satisfied: jupyter-client>=6.1.12 in  
 c:\users\surface\anaconda3\lib\site-packages (from nbclient>=0.5.0->nbconvert)  
 (8.6.3)  
 Requirement already satisfied: python-dateutil>=2.8.2 in  
 c:\users\surface\anaconda3\lib\site-packages (from jupyter-  
 client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)  
 Requirement already satisfied: pyzmq>=23.0 in  
 c:\users\surface\anaconda3\lib\site-packages (from jupyter-  
 client>=6.1.12->nbclient>=0.5.0->nbconvert) (26.2.0)  
 Requirement already satisfied: tornado>=6.2 in  
 c:\users\surface\anaconda3\lib\site-packages (from jupyter-  
 client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.5.1)  
 Requirement already satisfied: fastjsonschema>=2.15 in  
 c:\users\surface\anaconda3\lib\site-packages (from nbformat>=5.7->nbconvert)  
 (2.20.0)  
 Requirement already satisfied: jsonschema>=2.6 in  
 c:\users\surface\anaconda3\lib\site-packages (from nbformat>=5.7->nbconvert)  
 (4.23.0)  
 Requirement already satisfied: attrs>=22.2.0 in

```
c:\users\surface\anaconda3\lib\site-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (24.3.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
c:\users\surface\anaconda3\lib\site-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (2023.7.1)
Requirement already satisfied: referencing>=0.28.4 in
c:\users\surface\anaconda3\lib\site-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.30.2)
Requirement already satisfied: rpds-py>=0.7.1 in
c:\users\surface\anaconda3\lib\site-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.22.3)
Requirement already satisfied: six>=1.5 in c:\users\surface\anaconda3\lib\site-
packages (from python-dateutil>=2.8.2->jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.17.0)
Requirement already satisfied: soupsieve>1.2 in
c:\users\surface\anaconda3\lib\site-packages (from beautifulsoup4->nbconvert)
(2.5)
Note: you may need to restart the kernel to use updated packages.
```